ServiceNow + Microsoft Copilot AI Incident Routing

详细配置实施手册

第一部分：环境准备与前置配置

1.1 ServiceNow 插件安装与配置

1.1.1 必需插件清单

登录 ServiceNow 实例，导航至 **System Applications → All Available Applications → All**，安装以下插件：

插件名称	插件ID	用途	状态
Now Assist	com.snc.now_assist	AI核心平台	必需
Virtual Agent	com.glide.virtual_agent	对话机器人	必需
IntegrationHub	com.glide.hub.integrations	集成平台	必需
AI Search	com.snc.ai_search	语义搜索	必需
Microsoft Teams Integration	com.snc.teams_integration	Teams连接器	必需
Flow Designer	com.glide.flow_designer	工作流设计	必需

安装步骤：

```
1. 导航至: All > System Applications > All Available Applications > All
2. 搜索 "Now Assist" → 点击 Install
```

```
3. 等待安装完成（约5-10分钟）
4. 重复上述步骤安装其他插件
```

1.1.2 系统属性配置

导航至 **System Properties → All Properties**，配置以下属性：

属性名称	值	说明
`glide.virtual_agent.nlu.enabled`	`true`	启用NLU
`glide.now.assist.enabled`	`true`	启用Now Assist
`glide.ai_search.enabled`	`true`	启用AI Search

1.2 Azure OpenAI 服务配置

1.2.1 Azure 门户配置

步骤 1: 创建 Azure OpenAI Service 资源

```
1. 登录Azure Portal (https://portal.azure.com)
2. 点击 "Create a resource"
3. 搜索 "Azure OpenAI" → 点击Create
4. 填写基本信息：
   - Subscription: 选择订阅
   - Resource group: 创建或使用现有 (如: servicenow-ai)
   - Region: East US / West Europe (根据就近原则)
   - Name: servicenow-openai-prod
   - Pricing tier: Standard S0
5. 点击Review + Create → Create
```

步骤 2: 部署模型

```
1. 进入创建的Azure OpenAI 资源
2. 点击 "Model deployments" → "Create new deployment"
3. 选择模型: GPT-4 或GPT-4o
4. 部署名称: gpt-4-servicenow
```

```
5. 版本: 选择最新稳定版
6. 点击 Create
```

步骤 3: 获取 API Key 和 Endpoint

```
1. 在 Azure OpenAI 资源页面，点击 "Keys and Endpoint"
2. 复制以下信息（保存到安全位置）：
   - KEY 1: sk-xxxxxxxxxxxxxxxxxxxxxxxx
   - Endpoint: https://servicenow-openai-prod.openai.azure.com/
   - Deployment name: gpt-4-servicenow
```

1.2.2 ServiceNow 中配置 Azure OpenAI 连接

步骤 1: 创建 REST Message

```
导航: All > System Web Services > REST Message
点击: New
```

填写以下信息：

字段	值
Name	Azure OpenAI Service
Endpoint	https://servicenow-openai-prod.openai.azure.com
Authentication	No authentication (在HTTP Request中使用Header)

步骤 2: 配置 HTTP Method

在 REST Message 记录中，滚动到 **HTTP Methods** 相关列表，点击 **New**：

```
Name: chat_completion
HTTP Method: POST
Endpoint: /openai/deployments/gpt-4-servicenow/chat/completions?api-version=2024-02-15
```

步骤 3: 配置 HTTP Headers

在 HTTP Method 记录中，添加 Headers：

Header Name	Value
Content-Type	application/json
api-key	${azure.api.key}

步骤 4: 配置 Request Body

```
{
  "messages": [
    {
      "role": "system",
      "content": "${system_prompt}"
    },
    {
      "role": "user",
      "content": "${user_prompt}"
    }
  ],
  "temperature": 0.3,
  "max_tokens": 800
}
```

步骤 5: 配置 Variables

在 REST Message 中定义变量：

Variable Name	Value
azure.api.key	[从Azure复制的Key]
system_prompt	You are an ITSM routing assistant...
user_prompt	[动态传入]

第二部分：CI-KB 关联与数据模型配置

2.1 CMDB CI 字段扩展

2.1.1 添加 Support Group 字段到 CI

步骤 1: 导航至 **System Definition → Tables**

步骤 2: 搜索表 **Configuration Item [cmdb_ci]**

步骤 3: 在 **Columns** 相关列表中，点击 **New**：

```
Column Label: Support Group
Column Name: support_group
Type: Reference
Reference: Group [sys_user_group]
Max length: 32
Active: true
```

步骤 4: 添加表单字段

```
导航: All > Configuration > Servers > All Servers (或其他CI类型)
选择任意CI记录 → 右键 Form > Configure > Form Layout
将 "Support Group" 字段从左侧拖到右侧表单
保存
```

2.1.2 批量导入 CI-Support Group 映射

准备导入数据 (Excel/CSV)：

cmdb_ci.name	support_group.name
SAP Production Server	SAP Team
Oracle Database Server	DBA Team
Network Switch 01	Network Operations

导入步骤：

```
1. 导航: All > System Import Sets > Load Data
2. Source: File (上传CSV)
3. Target Table: Configuration Item [cmdb_ci]
4. Transform Map: 创建新的Transform Map
   - Source field: cmdb_ci.name → Target: name
   - Source field: support_group.name → Target: support_group (使用Reference Qualifier)
5. 运行Transform
```

2.2 KB 与 CI 关联配置

2.2.1 启用 CI 关联功能

步骤 1: 导航 **Knowledge → Administration → Properties**

步骤 2: 启用以下属性：

```
glide.know.use.cmdb_ci = true
glide.know.show.cmdb_ci = true
```

2.2.2 配置 KB 表单

步骤 1: 导航 **Knowledge → All**

步骤 2: 打开任意 KB Article → 右键 **Configure → Form Layout**

步骤 3: 添加字段： - **Configuration Item**: 从左侧拖到右侧 - **Assignment Group**: 拖到右侧（如果还没有）

2.2.3 创建 KB 分类标准

步骤 1: 导航 **Knowledge → Administration → Categories**

步骤 2: 创建以下分类结构：

```
IT Operations
├── Server Issues
```

```
│   ├── Hardware Failures
│   └── OS Problems
├── Database Issues
│   ├── Oracle
│   └── SQL Server
├── Network Issues
│   ├── Connectivity
│   └── Performance
└── Application Issues
    ├── SAP
    └── Custom Apps
```

步骤 3: 为每个分类配置 **Owner Group**（用于默认assignment）

第三部分：AI 路由引擎开发

3.1 Script Include: AIRoutingEngine

3.1.1 创建 Script Include

步骤 1: 导航 **System Definition → Script Includes → New**

步骤 2: 填写基本信息：

```
Name: AIRoutingEngine
API Name: AIRoutingEngine
Client callable: false
Active: true
Application scope: Global
```

步骤 3: 粘贴以下代码：

```
var AIRoutingEngine = Class.create();
AIRoutingEngine.prototype = {
    initialize: function() {
        this.restMessageName = 'Azure OpenAI Service';
```

```
        this.httpMethod = 'chat_completion';
        this.maxRetries = 3;
        this.timeout = 30000; // 30 seconds
    },

    /**
     * 主函数：获取 Assignment Group 推荐
     * @param {string} incidentId - Incident sys_id
     * @returns {Object} recommendation object
     */
    getAssignmentRecommendation: function(incidentId) {
        try {
            // 1. 获取 Incident 数据
            var incident = this._getIncidentData(incidentId);
            if (!incident) {
                return this._createErrorResponse('Incident not found');
            }

            // 2. 检索相关 KB Articles
            var kbArticles = this._retrieveKBArticles(incident.cmdb_ci);

            // 3. 构建 Prompt
            var prompt = this._buildPrompt(incident, kbArticles);

            // 4. 调用 Azure OpenAI
            var aiResponse = this._callAzureOpenAI(prompt);
            if (aiResponse.error) {
                return this._createErrorResponse(aiResponse.error);
            }

            // 5. 解析响应
            var recommendation = this._parseAIResponse(aiResponse, incident);

            // 6. 记录日志
            this._logRecommendation(incidentId, recommendation);

            return recommendation;

        } catch (e) {
            gs.error('AIRoutingEngine Error: ' + e.message);
            return this._createErrorResponse(e.message);
```

```
        }
    },

    /**
     * 获取Incident 完整数据
     */
    _getIncidentData: function(incidentId) {
        var gr = new GlideRecord('incident');
        if (!gr.get(incidentId)) {
            return null;
        }

        var ciSysId = gr.getValue('cmdb_ci');
        var ciName = '';
        var supportGroup = '';

        if (ciSysId) {
            var ci = new GlideRecord('cmdb_ci');
            if (ci.get(ciSysId)) {
                ciName = ci.getValue('name');
                var sg = ci.getValue('support_group');
                if (sg) {
                    var group = new GlideRecord('sys_user_group');
                    if (group.get(sg)) {
                        supportGroup = group.getValue('name');
                    }
                }
            }
        }

        return {
            sys_id: incidentId,
            number: gr.getValue('number'),
            short_description: gr.getValue('short_description'),
            description: gr.getValue('description'),
            category: gr.getValue('category'),
            subcategory: gr.getValue('subcategory'),
            priority: gr.getValue('priority'),
            urgency: gr.getValue('urgency'),
            impact: gr.getValue('impact'),
            cmdb_ci: ciSysId,
```

```
            ci_name: ciName,
            ci_support_group: supportGroup,
            assigned_to: gr.getValue('assigned_to'),
            assignment_group: gr.getValue('assignment_group')
        };
    },

    /**
     * RAG: 检索KB Articles
     */
    _retrieveKBArticles: function(ciSysId) {
        var articles = [];

        if (!ciSysId) {
            return articles;
        }

        // 方法1: 通过AI Search (如果可用)
        try {
            articles = this._searchWithAISearch(ciSysId);
        } catch (e) {
            gs.warn('AI Search failed, falling back to direct query: ' + e.message);
            articles = this._searchWithDirectQuery(ciSysId);
        }

        return articles;
    },

    /**
     * 使用AI Search (Semantic Search)
     */
    _searchWithAISearch: function(ciSysId) {
        var articles = [];

        // 获取CI详细信息用于语义搜索
        var ciInfo = this._getCIInfoForSearch(ciSysId);

        // 使用ServiceNow AI Search API
        var searchRequest = {
            search_terms: ciInfo.name + ' ' + ciInfo.model_id + ' incident handling',
            table: 'kb_knowledge',
```

```
            limit: 3,
            semantic_search: true
        };

        // 注意: 实际实现可能需要使用Scripted REST API 或Flow Designer
        // 这里使用简化版本
        var gr = new GlideRecord('kb_knowledge');
        gr.addQuery('cmdb_ci', ciSysId);
        gr.addQuery('workflow_state', 'published');
        gr.addActiveQuery();
        gr.orderByDesc('sys_updated_on');
        gr.setLimit(3);
        gr.query();

        while (gr.next()) {
            articles.push({
                number: gr.getValue('number'),
                title: gr.getValue('short_description'),
                content: gr.getValue('text').substring(0, 1000), // 限制长度
                category: gr.getDisplayValue('kb_category'),
                sys_id: gr.getValue('sys_id')
            });
        }

        return articles;
    },

    /**
     * 直接查询 (Fallback)
     */
    _searchWithDirectQuery: function(ciSysId) {
        var articles = [];

        var gr = new GlideRecord('kb_knowledge');
        gr.addQuery('cmdb_ci', ciSysId);
        gr.addQuery('workflow_state', 'published');
        gr.addActiveQuery();
        gr.orderByDesc('sys_updated_on');
        gr.setLimit(3);
        gr.query();
```

```
        while (gr.next()) {
            articles.push({
                number: gr.getValue('number'),
                title: gr.getValue('short_description'),
                content: gr.getValue('text').substring(0, 1000),
                category: gr.getDisplayValue('kb_category'),
                sys_id: gr.getValue('sys_id')
            });
        }

        return articles;
    },

    /**
     * 获取CI 信息用于搜索
     */
    _getCIInfoForSearch: function(ciSysId) {
        var info = {
            name: '',
            model_id: '',
            class_name: ''
        };

        var gr = new GlideRecord('cmdb_ci');
        if (gr.get(ciSysId)) {
            info.name = gr.getValue('name');
            info.model_id = gr.getValue('model_id');
            info.class_name = gr.getClassDisplayValue();
        }

        return info;
    },

    /**
     * 构建Prompt
     */
    _buildPrompt: function(incident, kbArticles) {
        var systemPrompt = "You are an expert IT Service Management routing assistant.
            "Analyze the incident details and KB articles to recommend the best assign
            "Consider the CI type, incident category, and KB article guidance. " +
            "Respond in JSON format with: assignment_group, confidence (high/medium/lo
```

```
        var userPrompt = "Incident Details:\n" +
            "Number: " + incident.number + "\n" +
            "Short Description: " + incident.short_description + "\n" +
            "Description: " + incident.description + "\n" +
            "Category: " + incident.category + "\n" +
            "Subcategory: " + incident.subcategory + "\n" +
            "Priority: " + incident.priority + "\n" +
            "Affected CI: " + incident.ci_name + "\n" +
            "CI Default Support Group: " + incident.ci_support_group + "\n\n";

        if (kbArticles.length > 0) {
            userPrompt += "Relevant KB Articles:\n";
            for (var i = 0; i < kbArticles.length; i++) {
                userPrompt += (i + 1) + ". " + kbArticles[i].number + " - " + kbArticl
                userPrompt += "Category: " + kbArticles[i].category + "\n";
                userPrompt += "Content: " + kbArticles[i].content.substring(0, 500) +
            }
        }

        userPrompt += "\nBased on this information, recommend the assignment group in
            "{\n" +
            "  \"assignment_group\": \"Group Name\",\n" +
            "  \"confidence\": \"high/medium/low\",\n" +
            "  \"reasoning\": \"explanation\"\n" +
            "}";

        return {
            system: systemPrompt,
            user: userPrompt
        };
    },

    /**
     * 调用Azure OpenAI
     */
    _callAzureOpenAI: function(prompt) {
        var r = new sn_ws.RESTMessageV2(this.restMessageName, this.httpMethod);

        // 设置变量
        r.setStringParameter('system_prompt', prompt.system);
```

```
        r.setStringParameter('user_prompt', prompt.user);

        // 设置超时
        r.setHttpTimeout(this.timeout);

        var response;
        var retries = 0;

        while (retries < this.maxRetries) {
            try {
                response = r.execute();
                var httpStatus = response.getStatusCode();

                if (httpStatus == 200) {
                    var body = response.getBody();
                    return JSON.parse(body);
                } else {
                    retries++;
                    if (retries >= this.maxRetries) {
                        return { error: 'HTTP ' + httpStatus + ': ' + response.getErro
                    }
                    // 等待1秒后重试
                    this._sleep(1000);
                }
            } catch (e) {
                retries++;
                if (retries >= this.maxRetries) {
                    return { error: e.message };
                }
                this._sleep(1000);
            }
        }

        return { error: 'Max retries exceeded' };
    },

    /**
     * 解析AI 响应
     */
    _parseAIResponse: function(aiResponse, incident) {
        try {
```

```
            // 提取content 从OpenAI 响应
            var content = '';
            if (aiResponse.choices && aiResponse.choices.length > 0) {
                content = aiResponse.choices[0].message.content;
            }

            // 尝试解析JSON
            var parsed;
            try {
                parsed = JSON.parse(content);
            } catch (e) {
                // 如果不是纯JSON，尝试提取JSON部分
                var jsonMatch = content.match(/\{[\s\S]*\}/);
                if (jsonMatch) {
                    parsed = JSON.parse(jsonMatch[0]);
                } else {
                    throw new Error('Invalid JSON response');
                }
            }

            // 验证assignment_group
            var groupSysId = this._validateAndGetGroup(parsed.assignment_group);

            return {
                success: true,
                assignment_group_name: parsed.assignment_group,
                assignment_group_sysid: groupSysId,
                confidence: parsed.confidence || 'medium',
                reasoning: parsed.reasoning || 'No reasoning provided',
                original_ci_support_group: incident.ci_support_group,
                ai_recommended: groupSysId !== incident.assignment_group
            };

        } catch (e) {
            return this._createErrorResponse('Failed to parse AI response: ' + e.messa
        }
    },

    /**
     * 验证并获取Group Sys ID
     */
```

```
    _validateAndGetGroup: function(groupName) {
        if (!groupName) return null;

        var gr = new GlideRecord('sys_user_group');
        gr.addQuery('name', groupName);
        gr.addActiveQuery();
        gr.query();

        if (gr.next()) {
            return gr.getValue('sys_id');
        }

        return null;
    },

    /**
     * 记录推荐日志
     */
    _logRecommendation: function(incidentId, recommendation) {
        gs.info('AI Routing Recommendation for ' + incidentId + ': ' +
            JSON.stringify(recommendation));

        // 可以扩展到写入自定义日志表
    },

    /**
     * 创建错误响应
     */
    _createErrorResponse: function(errorMessage) {
        return {
            success: false,
            error: errorMessage,
            assignment_group_name: null,
            assignment_group_sysid: null,
            confidence: 'low',
            reasoning: 'AI recommendation failed: ' + errorMessage
        };
    },

    /**
     * 辅助：睡眠函数
```

```
     */
    _sleep: function(ms) {
        var end = new Date().getTime() + ms;
        while (new Date().getTime() < end) {
            // 忙等待
        }
    },

    type: 'AIRoutingEngine'
};
```

步骤 4: 点击 **Submit** 保存

3.2 Business Rule: 自动触发 AI 路由

3.2.1 创建 After Insert Business Rule

步骤 1: 导航 **System Definition → Business Rules → New**

步骤 2: 填写基本信息：

```
Name: AI Routing - Auto Assignment
Table: Incident [incident]
Active: true
Advanced: true
When: after
Insert: true
Update: false
Filter Conditions:
  - Configuration item is not empty
  - Assignment group is empty
```

步骤 3: 在 **Advanced** 标签页粘贴脚本：

```
(function executeRule(current, previous /*null when async*/) {

    // 避免递归触发
    if (current.getValue('u_ai_processing') == 'true') {
```

```
        return;
    }

    try {
        // 调用AI 路由引擎
        var aiRouter = new AIRoutingEngine();
        var recommendation = aiRouter.getAssignmentRecommendation(current.getValue('sy

        if (recommendation.success && recommendation.assignment_group_sysid) {

            // 设置标记避免递归
            current.setValue('u_ai_processing', 'true');

            // 更新Assignment Group
            current.setValue('assignment_group', recommendation.assignment_group_sysid

            // 添加工作记录
            var workNote = '[AI Routing] Recommended assignment: ' +
                recommendation.assignment_group_name +
                ' (Confidence: ' + recommendation.confidence + ')\n' +
                'Reasoning: ' + recommendation.reasoning;

            current.setWorkNote(workNote);

            gs.info('AI Routing applied to ' + current.getValue('number') +
                ': ' + recommendation.assignment_group_name);
        } else {
            // AI 推荐失败，使用CI 默认Support Group
            var ciSysId = current.getValue('cmdb_ci');
            if (ciSysId) {
                var ci = new GlideRecord('cmdb_ci');
                if (ci.get(ciSysId) && ci.getValue('support_group')) {
                    current.setValue('assignment_group', ci.getValue('support_group'))
                    current.setWorkNote('[AI Routing] Fallback to CI Support Group. AI
                        (recommendation.error || 'Unknown'));
                }
            }
        }

    } catch (e) {
        gs.error('AI Routing Business Rule Error: ' + e.message);
```

```
    }

})(current, previous);
```

步骤 4: 点击 **Submit**

3.2.2 创建字段标记

步骤 1: 导航 **System Definition → Tables → Incident**

步骤 2: 在 **Columns** 中新建：

```
Column Label: AI Processing
Column Name: u_ai_processing
Type: True/False
Default value: false
Active: true
```

第四部分：P1 通知工作流配置

4.1 Notification Rules 配置

4.1.1 创建 P1 Incident Notification

步骤 1: 导航 **System Notification → Notifications → New**

步骤 2: 填写基本信息：

```
Name: P1 Incident Alert - On-Call Engineer
Table: Incident [incident]
Active: true
Category: Incident Management
```

步骤 3: 配置 **When to send**：

```
Send when: Record inserted or updated
Condition: Priority is 1 - Critical
           AND State is New
```

步骤 4: 配置 **Who will receive**：

```
User/Group: [Script]
```

在 **Recipient Script** 中：

```
(function runRecipientScript(current) {
    var recipients = [];

    // 获取Assignment Group 的On-Call 成员
    var groupId = current.getValue('assignment_group');
    if (groupId) {
        // 使用On-Call Scheduling API
        var onCallGR = new GlideRecord('cmn_rota');
        onCallGR.addQuery('group', groupId);
        onCallGR.addActiveQuery();
        onCallGR.query();

        while (onCallGR.next()) {
            var member = onCallGR.getValue('member');
            if (member) {
                recipients.push(member);
            }
        }
    }

    // 如果找不到on-call，发送给group manager
    if (recipients.length === 0 && groupId) {
        var group = new GlideRecord('sys_user_group');
        if (group.get(groupId) && group.getValue('manager')) {
            recipients.push(group.getValue('manager'));
        }
    }
```

```
    return recipients;
})(current);
```

4.1.2 配置通知内容

步骤 1: 在 **What it will contain** 中：

```
Subject: [P1 CRITICAL] ${number} - ${short_description}

Message HTML:
```

```
<div style="font-family: Arial, sans-serif; padding: 20px;">
    <div style="background-color: #d32f2f; color: white; padding: 15px; border-radius:
        <h2 style="margin: 0;">□ P1 CRITICAL INCIDENT</h2>
    </div>

    <div style="margin-top: 20px;">
        <p><strong>Incident Number:</strong> ${number}</p>
        <p><strong>Short Description:</strong> ${short_description}</p>
        <p><strong>Priority:</strong> ${priority}</p>
        <p><strong>Affected CI:</strong> ${cmdb_ci}</p>
        <p><strong>Assignment Group:</strong> ${assignment_group}</p>
        <p><strong>Opened:</strong> ${opened_at}</p>
        <p><strong>Opened By:</strong> ${opened_by}</p>
    </div>

    <div style="margin-top: 20px; padding: 15px; background-color: #fff3e0; border-lef
        <p><strong>Description:</strong></p>
        <p>${description}</p>
    </div>

    <div style="margin-top: 20px;">
        <a href="${instance_url}/nav_to.do?uri=incident.do?sys_id=${sys_id}"
           style="background-color: #1976d2; color: white; padding: 10px 20px; text-de
            View Incident
        </a>
    </div>
</div>
```

4.2 Flow Designer: P1 Escalation Workflow

4.2.1 创建 Flow

步骤 1: 导航 **Flow Designer**

步骤 2: 点击 **New → Flow**

```
Flow Name: P1 Incident Escalation
Run: Always
Run As: System User
```

4.2.2 配置 Trigger

添加 Trigger: - **Table**: Incident - **Condition**: Priority is 1 - Critical - **Run**: When record is created

4.2.3 添加 Actions

Action 1: Wait for duration

```
Duration: 15 minutes
```

Action 2: Check if incident is still open

```
Condition: State is not Resolved
          AND State is not Closed
```

Action 3: If condition true - Escalate

```
Action: Send Email
To: [Assignment Group Manager]
Subject: [ESCALATION] P1 ${number} unresolved after 15 minutes
```

Action 4: Create Slack Message

```
Action: Slack - Post Message
Channel: #incidents-critical
Message: □ P1 Escalation: ${number} needs attention
```

步骤 4: 保存并激活 Flow

第五部分：Microsoft Copilot 集成

5.1 Azure AD 应用注册

5.1.1 创建应用注册

步骤 1: 登录 **Azure Portal → Azure Active Directory → App registrations → New registration**

步骤 2: 填写信息：

```
Name: ServiceNow-Copilot-Integration
Supported account types: Accounts in this organizational directory only
Redirect URI (optional): https://your-instance.service-now.com/oauth_redirect.do
```

步骤 3: 点击 **Register**

步骤 4: 记录以下信息：

```
Application (client) ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
Directory (tenant) ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

5.1.2 添加客户端密钥

步骤 1: 在应用页面，点击 **Certificates & secrets → New client secret**

步骤 2: 填写描述并选择过期时间

步骤 3: 立即复制密钥值（只显示一次）

5.1.3 配置 API 权限

步骤 1: 点击 **API permissions → Add a permission**

步骤 2: 添加以下权限：

```
Microsoft Graph:
  - User.Read
  - Chat.Read
  - Chat.Create
  - TeamsAppInstallation.ReadForUser

Azure Service Management:
  - user_impersonation
```

步骤 3: 点击 **Grant admin consent**

5.2 ServiceNow OAuth 配置

5.2.1 创建 OAuth Provider

步骤 1: 导航 **System OAuth → Application Registry → New**

步骤 2: 选择 **Connect to a third party OAuth Provider**

步骤 3: 填写信息：

```
Name: Microsoft Teams Copilot
Client ID: [从Azure复制的Client ID]
Client Secret: [从Azure复制的Client Secret]
Token URL: https://login.microsoftonline.com/[tenant-id]/oauth2/v2.0/token
Authorization URL: https://login.microsoftonline.com/[tenant-id]/oauth2/v2.0/authorize
Default Grant Type: Authorization Code
Redirect URL: https://your-instance.service-now.com/oauth_redirect.do
```

5.3 Teams Integration 配置

5.3.1 安装 Teams Integration 插件

已在 **1.1.1** 中安装 `com.snc.teams_integration`

5.3.2 配置 Teams 连接

步骤 1: 导航 **Microsoft Teams Integration → Setup**

步骤 2: 点击 **Configure**

步骤 3: 授权 Teams 连接

5.3.3 创建 Custom Engine Agent (Zurich+)

注意: 如果实例是 **Zurich 版本**，可以使用 Custom Engine Agent：

步骤 1: 导航 **Conversational Interfaces → Settings → Microsoft Copilot**

步骤 2: 启用 **Custom Engine Agent**

步骤 3: 配置 Bot 名称和 Manifest

5.4 Virtual Agent Topic 配置

5.4.1 创建 NLU Model

步骤 1: 导航 **NLU Workbench → Models → New**

```
Name: Incident Routing Model
Language: English (or your language)
```

步骤 2: 创建 Intents：

Intent	Description	Sample Utterances
assignment_query	查询assignment建议	"Who should handle this incident?", "What team should I assign this to?"

Intent	Description	Sample Utterances
kb_search	搜索KB	"Find KB articles for this issue", "Show me documentation"
escalate_incident	升级incident	"I need to escalate this", "This is urgent"

5.4.2 创建 Virtual Agent Topic

步骤 1: 导航 **Virtual Agent → Topics → New**

```
Name: Incident Assignment Assistant
NLU Model: Incident Routing Model
Trigger Intent: assignment_query
```

步骤 2: 配置 Topic Flow：

```
1. Start
2. Ask for: Incident Number (or use current context)
3. Script Action: Call AIRoutingEngine.getAssignmentRecommendation()
4. Display: AI recommendation with confidence
5. Ask: "Would you like me to update the assignment?"
6. If Yes → Update Incident → Confirmation message
7. If No → End
```

步骤 3: 在 **Script Action** 中：

```
(function execute() {
    var incidentNumber = conversation.variables.incident_number;

    // 获取incident sys_id
    var gr = new GlideRecord('incident');
    gr.addQuery('number', incidentNumber);
    gr.query();

    if (gr.next()) {
        var aiRouter = new AIRoutingEngine();
        var rec = aiRouter.getAssignmentRecommendation(gr.getValue('sys_id'));
```

```
        conversation.variables.recommended_group = rec.assignment_group_name;
        conversation.variables.confidence = rec.confidence;
        conversation.variables.reasoning = rec.reasoning;
        conversation.variables.group_sysid = rec.assignment_group_sysid;
    } else {
        conversation.variables.error = "Incident not found";
    }
})();
```

第六部分：测试与验证

6.1 单元测试

6.1.1 测试 Script Include

步骤 1: 导航 **System Definition → Scripts - Background**

步骤 2: 粘贴测试代码：

```
// 测试 AIRoutingEngine
var testIncidentId = '[一个测试incident的sys_id]';
var aiRouter = new AIRoutingEngine();
var result = aiRouter.getAssignmentRecommendation(testIncidentId);

gs.info('Test Result: ' + JSON.stringify(result, null, 2));
```

步骤 3: 点击 **Run script**

6.1.2 测试 Business Rule

步骤 1: 创建测试 Incident

步骤 2: 设置 Configuration Item（关联到 CI）

步骤 3: 保存并观察： - Assignment Group 是否自动更新 - Work Notes 是否添加了 AI 推荐记录

6.2 集成测试

6.2.1 测试 Azure OpenAI 连接

```
var r = new sn_ws.RESTMessageV2('Azure OpenAI Service', 'chat_completion');
r.setStringParameter('system_prompt', 'You are a helpful assistant');
r.setStringParameter('user_prompt', 'Hello, are you working?');

var response = r.execute();
var body = response.getBody();
gs.info('Response: ' + body);
```

6.2.2 测试 P1 通知

步骤 1: 创建 P1 Incident

步骤 2: 验证： - Email 是否发送给 On-Call 工程师 - SMS 是否触发（如果配置了） - Teams 消息是否发送

6.2.3 测试 Virtual Agent

步骤 1: 打开 Service Portal → Virtual Agent

步骤 2: 输入："Who should handle INC0012345?"

步骤 3: 验证： - Virtual Agent 是否识别 Intent - 是否返回正确的 Assignment Group 推荐 - 是否可以一键更新 Assignment

---

6.3 性能优化

6.3.1 添加缓存机制

如果 AI 调用频繁，可以添加 GlideCache：

```
// 在 AIRoutingEngine 中添加缓存
_getCachedRecommendation: function(incidentHash) {
    var cache = new GlideCache();
```

```
    cache.setTimeOut(300); // 5分钟缓存
    return cache.get(incidentHash);
},

_setCachedRecommendation: function(incidentHash, recommendation) {
    var cache = new GlideCache();
    cache.set(incidentHash, recommendation);
}
```

6.3.2 异步处理

如果 AI 调用慢，使用 Async Business Rule：

```
When: async
Insert: true
Advanced: true
```

附录：故障排除

A.1 常见问题

Q: Azure OpenAI 返回 401 Unauthorized

A: 检查 REST Message 中的 api-key header 是否正确

Q: Business Rule 不触发

A: 1. 检查 Business Rule 是否 Active 2. 检查 Condition 是否正确 3. 检查是否有其他 BR 冲突

Q: AI 推荐不准确

A: 1. 检查 KB Articles 质量和数量 2. 优化 Prompt 中的 instructions 3. 考虑 fine-tuning (需要额外 Azure OpenAI 配置)

Q: Teams 集成失败

A: 1. 检查 OAuth 配置 2. 验证 Azure AD 权限是否 granted 3. 检查 Teams Integration 插件是否最新

A.2 监控与日志

启用 DEBUG 日志

```
// 在Script Include 中添加
this.debug = true;

// 在关键位置
if (this.debug) {
    gs.debug('Debug: ' + JSON.stringify(data));
}
```

查看系统日志

```
All > System Logs > System Log
Filter: Source contains "AIRoutingEngine"
```

文档完成时间: 2026-03-08 **版本**: v1.0 **作者**: 富贵虾 (AI Architect)